

create_bound

NAME

create_bound
Creates a move bound or group bound in the current design.

SYNTAX

```
collection create_bound
  -name bound_name
  [-boundary regions]
  [-dimensions bound_dimension]
  [-diamond central_object]
  [-effort low | medium | high | ultra]
  [-type soft | hard]
  [-exclusive]
  [-macro_group]
  [-hierarchical_only]
  [-design_type design_type]
  [-repelling diamond | rect]
  [-cell cell]
  [cell_and_port_and_group_list]
  [-color color]
```

Data Types

bound_name	string
regions	list
bound_dimension	list
central_object	collection
design_type	string
cell	collection
cell_and_port_and_group_list	list
color	string

ARGUMENTS

-name bound_name
Specifies the name of the bound.

-boundary regions
Creates move bounds with the specified boundaries. You can specify one or more rectangles, polygons, and geometric objects. If you specify more than one placeable area, the coarse placer can place cells in any of the move bound's bound shapes. These placeable areas can overlap or be disjoint.

To specify a rectangle, use the following format to specify the lower-left and upper-right corners of the rectangle:
{llx lly} {urx ury}

To specify a polygon, use the following format to specify the coordinates of the polygon:
{x1 y1} {x2 y2} {x3 y3} ... {xn yn}

Each rectangle and polygon defines a target placement area for the objects. The areas can overlap or be disjoint. The coordinates are relative to the chip origin.

You can also specify polygons as the combined area of a heterogeneous collection of objects with physical geometry, such as poly_rects, geo_masks, shapes, layers, and other physical objects. For poly_rects, geo_masks, shapes, or other physical objects, the resulting area includes the areas of each object. For layers, the resulting area includes the area of every shape in the layer.

This option is mutually exclusive with the -dimensions option; you can specify only one.

-dimensions bound_dimension
Creates a group bound with the specified dimensions.

For a group bound, specify the dimension as {width height}.

For a diamond bound, specify the dimension as {length}. When you use this format, you must also specify the -diamond option.

This option is mutually exclusive with the -boundary option; you can specify only one.

-diamond central_object
 Creates a diamond bound centered on the specified object, which can be a port, cell, or pin. You can specify only one object.

The objects in the bound are constrained to lie within the distance specified by the **-dimensions** option (measured as a Manhattan distance) of the specified object. A diamond bound is always a soft bound.

If you specify this option, you must also specify the **-dimensions** option.

-effort low | medium | high | ultra
 Specifies how strongly a bound works to keep cells within the constraint.

When you set the effort to ultra (the default), cells will have limited ability to be placed outside the bound shape. Lower effort levels allow cells to be placed further from the bound. Lowering effort is useful when you want to allow placement flexibility to fix timing, wire length, or congestion. Most large bounds work well with medium effort.

Bounds with **-boundary** or **-dimensions** have a default effort of ultra. Otherwise, the default effort is medium.

You may not specify **-effort** with the **-diamond**, **-repelling**, **-exclusive**, or **-type hard** options.

-type soft | hard
 Sets the bound type to be either hard or soft.

The default is soft.

When you specify **-type hard**,

- o You must also specify the **-boundary** or **-dimensions** option
- o You cannot specify the **-effort** option

-exclusive
 Sets the bound to be exclusive. This option is allowed only for move bounds. A move bound that is exclusive requires all of its cells to be placed inside it and prohibits the placement of other cells in the same area. Exclusive move bounds are respected by both coarse placement and legalization.

-macro_group
 Sets the bound to be macro_group bound. This option is allowed only for move bounds.

This option specifies that the bound can contain only cells whose reference is of macro design type.

This option is mutually exclusive with **-exclusive**, **-hierarchical_only** and **-design_type** options.

-hierarchical_only
 Specifies that the bound can contain only cells that represent module boundary objects, which are created by the **explore_logic_hierarchy** command. This option is allowed only for move bounds. No other cells or ports can be added to move bounds of this type.

This option creates move bounds that are used during floorplanning; they have no effect on other placement or legalization.

-design_type design_type
 Specifies that the bound can contain only cells whose reference is of the specified design type. This option is allowed only for move bounds. The valid values for this option are **abstract**, **analog**, **black_box**, **corner**, **cover**, **diode**, **end_cap**, **feedthrough**, **fill**, **filler**, **flip_chip_driver**, **flip_chip_pad**, **lib_cell**, **macro**, **module**, **pad**, **pad_spacer**, and **well_tap**. No other cells or ports can be added to move bounds of this type.

This option creates move bounds that are used during floorplanning; they have no effect on other placement or legalization. The expected usage is to create move bounds that contain only hard macros.

-repelling diamond | rect
 Specifies that the bound is repelling rather than attractive.

Repelling bounds keep the bound contents spread apart by the specified shape region.

`-cell cell`

Specifies the physical cell in which to add the bound. The bound is created in the cell's reference block using the coordinate system of the cell's top block. The cell must reference a block, not a library cell, unless this command is run in the library manager. In the library manager, bounds can be created in library cells.

If you do not specify this option, the bound is created in the current block.

`cell_and_port_and_group_list`

Specifies the cells, ports, and groups to assign to the bound. The list can contain names, patterns, or collections. Group objects are only valid when creating a repelling bound. Furthermore, repelling bounds may contain either cells and ports or groups, but not a mix.

It is an error to specify a port that has already been assigned to another bound.

If this bound is a move bound, it is an error to specify a cell that has already been explicitly assigned to another move bound.

If this bound is a group bound, it is an error to specify a cell that has already been explicitly assigned to another group bound.

If a cell is hierarchical, its child cells are also implicitly in the bound, as well as any cells added to its hierarchy in the future, unless the child cell is assigned to another bound.

Cells that belong to a relative placement group cannot be individually assigned to a bound. All cells of a relative placement group must be assigned to a bound together.

If you do not specify this argument, the tool creates an empty bound. You can later assign cells and ports to the bound by using the `add_to_bound` command.

`-color color`

Creates a move bound with the specified color, where color is a 24-bit RGB hexadecimal string prefaced by '#', for example: "#72c4f1" or "#ce8322".

DESCRIPTION

This command defines region-based placement constraints for coarse placement. There are two different types of bounds:

o Move bound

A move bound restricts the placement of cells within fixed regions (bound shapes) of the core area. To create a move bound, specify the absolute coordinates with the `-boundary` option.

o Group bound

A group bound restricts the placement of cells within a floating region. Cells are placed within a bound but its absolute coordinates are not fixed, instead they are optimized by the placer.

To specify the floating region size for a group bound, use the `-dimensions` option. If you do not specify the `-dimensions` (or `-boundary`) option, the tool creates an automatic group bound, whose floating region is automatically determined by the placer.

Repelling bounds function like group bounds, but instead of defining an area within which the objects should be placed, they indicate an area of separation that should exist between each pair of objects in the bound.

Generally, there is no guarantee that cells will be placed completely within a bound. For example, coarse placement will violate bounds if the quality of its primary placement objectives would otherwise be degraded. In these situations, you should revisit your and floorplan to ensure that the block is not overconstrained. Alternatively, you can use the `-type hard` option to create a hard bound (the default is soft). The coarse placer tries to honor hard bounds as hard constraints while sacrificing other objectives, such as timing and routability. You should not use many hard bounds because this might result in inferior

placement solutions.

To assign cells and ports to an existing bound, use the `add_to_bound` command. To unassign cells and ports from a bound, use the `remove_from_bound` command.

EXAMPLES

The following example creates a hard exclusive move bound named "move-bound1" with one rectangular and one polygon bound shape. The bound contains the `invA1` cell and `reset` port.

```
prompt> create_bound -name "movebound1" -boundary {{{100 100} {200 200}} \
  {{{1000 1000} {2000 1000} {2000 4000} {1500 4000} {1500 2000} {1000 2000}}}} \
  -type hard -exclusive {invA1 reset}
```

The following example creates a move bound named "movebound2" that contains the hierarchical cell named `mid`. It also implicitly contains all cells within the `mid` hierarchical cell.

```
prompt> create_bound -name "movebound2" -boundary {{0 0} {200 200}} \
  [get_cells mid]
```

The following example creates a move bound named "movebound3" with rectilinear bound shapes that have the same boundaries as the merged areas of voltage area shapes whose names start with `VA_SHP_`.

```
-boundary [get_voltage_area_shapes VA_SHP_*]
```

The following example creates an automatic group bound named "group-bound1" with ultra effort that contains all physical cells whose names start with `adder`.

```
prompt> create_bound -name "groupbound1" -effort ultra { adder* }
```

The following example creates a diamond bound named "temp3" centered on the `MOD/U1/A` pin. The `MOD/INST1` and `MOD/INST2` cells are constrained to be at most 64 microns of Manhattan distance away from `MOD/U1/A`.

```
prompt> create_bound -name "temp3" -dimensions 64 \
  -diamond MOD/U1/A {MOD/INST1 MOD/INST2}
```

The following example creates a macro_group move bound named "macroGroupMovebound".

```
prompt> create_bound -name "macroGroupMovebound" -macro_group -boundary {{0 0} {200 200}}
```

SEE ALSO

[add_to_bound\(2\)](#)
[create_bound_shape\(2\)](#)
[get_bound_shapes\(2\)](#)
[get_bounds\(2\)](#)
[remove_bound_shapes\(2\)](#)
[remove_bounds\(2\)](#)
[remove_from_bound\(2\)](#)
[report_bounds\(2\)](#)

Version V-2023.12-SP5-6

Copyright (c) 2025 Synopsys, Inc. All rights reserved.

Man(1) output converted with [man2html](#)